

UI Juicer - Complete Documentation

Comprehensive API reference and usage guide for the One-Click UI Animator

Table of Contents

 Overview

 Quick Start

 Methods

 Unity Events

 Feeling Presets

 Code Examples

 API Reference

 UiJuiceAnimator Class

 JuiceElement Class

 FeelingPreset Enum

 Properties



Overview

UI Juicer is a powerful Unity component that enables one-click UI animations with 60+ feeling presets

✨ What is UI Juicer?

UI Juicer is a Unity component that automatically animates UI elements with beautiful, staggered animations. It supports position, rotation, scale, fade, and color animations, all configurable through an intuitive inspector interface or code.

🎯 Key Features:

- 🎨 60+ feeling presets (Happy, Dramatic, Bounce, etc.)
- ⚡ 5 animation types: Position, Rotation, Scale, Fade, Color
- 🔄 Staggered/cascading animations
- 🎲 Auto-randomization for variation
- 🎮 Edit mode preview
- 📡 Unity Events support
- 🚀 No coding required (but code API available)

✅ **Ready to Use!** 🚀 UI Juicer is designed to work out of the box with zero code required. All features are accessible through the Unity Inspector. The code API is available for advanced users who want more control. 💪



Quick Start

Get up and running in seconds



Setup Steps

1. 📦 Add the `UiJuiceAnimator` component to your UI root GameObject (Canvas or Panel)
2. ▶️ Click "Play In" to see your UI animate!

```
// 💡 Optional: Control via code
public class MenuController : MonoBehaviour
{
    public UiJuiceAnimator juiceAnimator;

    void ShowMenu()
    {
        juiceAnimator.PlayIn();
    }

    void HideMenu()
    {
        juiceAnimator.PlayOut();
    }
}
```



Methods

All public methods and their usage

Method	Return Type	Description
<code>Play()</code>	<code>void</code>	 Plays the Play IN animation. Equivalent to <code>PlayIn()</code> .
<code>PlayIn()</code>	<code>void</code>	 Animates UI elements from their initial offset positions to their final positions. Automatically enables the GameObject if disabled.
<code>PlayOut()</code>	<code>void</code>	 Animates UI elements from their current positions to their offset positions (reverse of Play In).
<code>Stop()</code>	<code>void</code>	 Immediately stops all running animations and cleans up resources.
<code>RandomizeElements()</code>	<code>void</code>	 Re-applies the current preset with random variation to all elements. Useful for adding variety.
<code>ApplyFeelingPreset()</code>	<code>void</code>	 Applies the currently selected preset to all elements. Uses the <code>preset</code> property.
<code>ApplyFeelingPreset(FeelingPreset feeling)</code>	<code>void</code>	 Applies the specified feeling preset to all elements. Overrides the current <code>preset</code> property.
<code>RebuildFromHierarchy(bool includeInactive)</code>	<code>void</code>	 Scans all child RectTransforms and rebuilds the elements list. Set <code>includeInactive</code> to true to include inactive GameObjects.
<code>ResetToInitialState()</code>	<code>void</code>	 Resets all elements to their initial captured state. Useful for editor mode or manual resets.

Method Details

PlayIn()

This is the primary method for showing/animated UI elements. It:

-  Automatically enables the GameObject if it's disabled
-  Applies randomization if `autoRandomizeOnPlay` is enabled
-  Captures initial states of all elements
-  Starts staggered animations for all elements
-  Invokes `onAnimationStart` event

```
// 💡 Example: Show a menu
void OnMenuButtonClick()
{
    menuJuiceAnimator.PlayIn();
}
```

PlayOut()

Reverses the Play In animation, moving elements back to their offset positions. After completion:

-  Invokes `onAnimationFinish` event
-  In runtime: Can deactivate or destroy GameObject if configured
-  In editor: Automatically resets to initial state

```
// 💡 Example: Hide a menu
void OnCloseButtonClick()
{
    menuJuiceAnimator.PlayOut();
}
```

ApplyFeelingPreset(FeelingPreset feeling)

Applies a specific preset to all elements, configuring their animation parameters to match the preset's style.

```
// 💡 Example: Change preset dynamically
void OnMoodChanged(Mood newMood)
{
    switch (newMood)
    {
        case Mood.Happy:

juiceAnimator.ApplyFeelingPreset(UiJuiceAnimator.FeelingPreset.Ha
ppy);

        break;
        case Mood.Dramatic:

juiceAnimator.ApplyFeelingPreset(UiJuiceAnimator.FeelingPreset.Dr
amatic);

        break;
    }
}
```



Unity Events

Hook into animation lifecycle events

onAnimationStart

Type: `UnityEvent`

Invoked: When `PlayIn()` or `PlayOut()` is called, before animations begin.

```
// 🎧 Example: Play sound effect when animation starts
void Start()
{

    juiceAnimator.onAnimationStart.AddListener(OnAnimationStarted);
}

void OnAnimationStarted()
{
    audioSource.PlayOneShot(whooshSound);
}
```

onAnimationFinish

Type: `UnityEvent`

Invoked: When all animations complete (after the last element finishes).


```
// 📄 Example: Enable input after animation completes
void Start()
{

    juiceAnimator.onAnimationFinish.AddListener(OnAnimationFinished);
}

void OnAnimationFinished()
{
    inputEnabled = true;
    menuButtons.SetInteractable(true);
}
```

💡 **Pro Tip:** 🎯 You can also connect these events in the Unity Inspector without writing any code! Just drag your GameObject into the event slot and select the method to call. It's that easy! ✨



Feeling Presets

Complete list of all 60+ animation presets

Each preset configures position offsets, rotation, scale, fade, timing, and easing curves to create a unique animation feeling. Presets are applied via `ApplyFeelingPreset()` or by setting the `preset` property.

📄 **Note:** 🎨 Presets can be customized further using the `strength` property (multiplies offsets and intensity) and `durationMin` / `durationMax` (clamps timing). 💪



Code Examples

Real-world usage examples



Example 1: Menu Controller

```
using UnityEngine;
using JuiceUp;

public class MenuController : MonoBehaviour
{
    [SerializeField] private UiJuiceAnimator menuAnimator;
    [SerializeField] private AudioSource audioSource;

    void Start()
    {
        //  Hook into events
        menuAnimator.onAnimationStart.AddListener(PlayMenuSound);
    }

    public void ShowMenu()
    {
        if (!menuAnimator.IsAnimating)
        {
            menuAnimator.PlayIn();
        }
    }

    public void HideMenu()
    {
        if (!menuAnimator.IsAnimating)
        {
            menuAnimator.PlayOut();
        }
    }

    private void PlayMenuSound()
    {
        audioSource.Play();
    }
}
```

```
}  
}
```



Example 2: Dynamic Preset Switching

```
using UnityEngine;
using JuiceUp;

public class MoodBasedAnimator : MonoBehaviour
{
    public UiJuiceAnimator juiceAnimator;

    public void SetHappyMood()
    {
        juiceAnimator.preset =
        UiJuiceAnimator.FeelingPreset.Happy;
        juiceAnimator.strength = 1.2f;
        juiceAnimator.ApplyFeelingPreset();
    }

    public void SetDramaticMood()
    {
        juiceAnimator.preset =
        UiJuiceAnimator.FeelingPreset.Dramatic;
        juiceAnimator.strength = 1.5f;
        juiceAnimator.ApplyFeelingPreset();
    }

    public void SetCalmMood()
    {
        juiceAnimator.preset =
        UiJuiceAnimator.FeelingPreset.Calm;
        juiceAnimator.strength = 0.8f;
        juiceAnimator.ApplyFeelingPreset();
    }
}
```



Example 3: Random Preset on Each Show

```
using UnityEngine;
using JuiceUp;
using System;

public class RandomPresetAnimator : MonoBehaviour
{
    public UiJuiceAnimator juiceAnimator;

    public void ShowWithRandomPreset()
    {
        // 🎲 Get all preset values
        Array presetValues =
            Enum.GetValues(typeof(UiJuiceAnimator.FeelingPreset));

        // 🎯 Pick a random preset
        UiJuiceAnimator.FeelingPreset randomPreset =

            (UiJuiceAnimator.FeelingPreset)presetValues.GetValue(Random.Range
            (0, presetValues.Length));

        // ▶ Apply and play
        juiceAnimator.ApplyFeelingPreset(randomPreset);
        juiceAnimator.PlayIn();
    }
}
```


Example 4: Chaining Animations

```
using UnityEngine;
using JuiceUp;
using System.Collections;

public class AnimationSequence : MonoBehaviour
{
    public UiJuiceAnimator titleAnimator;
    public UiJuiceAnimator contentAnimator;
    public UiJuiceAnimator buttonAnimator;

    public void PlaySequence()
    {
        StartCoroutine(PlaySequenceCoroutine());
    }

    private IEnumerator PlaySequenceCoroutine()
    {
        // 📄 Play title
        titleAnimator.PlayIn();
        yield return new WaitForSeconds(0.5f);

        // 📄 Play content
        contentAnimator.PlayIn();
        yield return new WaitForSeconds(0.3f);

        // 🕒 Play buttons
        buttonAnimator.PlayIn();
    }
}
```



API Reference

Complete reference for all classes, methods, and properties



UiJuiceAnimator Class

Namespace: `JuiceUp`

Inherits: `MonoBehaviour`

Component Type: `DisallowMultipleComponent`

The main component class that handles UI animations. Attach this to your UI root GameObject to enable animations on child elements.



JuiceElement Class

Type: `[System.Serializable]`

Nested Class: Inside `UiJuiceAnimator`

Represents a single UI element and its animation configuration. Each element in the `elements` list contains settings for position, rotation, scale, fade, and color animations.

FeelingPreset Enum

Type: `public enum`

Default: `Dramatic`

Defines the animation style/feeling to apply. Each preset configures offsets, curves, timing, and easing to match its emotional tone.

Available Presets (60+):

Happy

Scary

Slow

Shaky

Calm

Energetic

Bounce

Gentle

Dramatic

Snappy

Heavy

Pop

Drift

Punchy

Rubber

SoftRise

Flashy

SlideLeft

SlideRight

Flip

ZoomPop

Wobble

Ripple

Stomp

FloatUp

SwoopIn

Blink

Cascade

Jitter

PulseGlow

TiltDrop

SnapLeft

SnapRight

BounceInPlace

DriftSlow

PopRotate

Whip

Wave

Breath

FlashSlideDown

SlideOutLeft

SlideOutRight

SlideOutTop

SlideOutDown

FadeOut

FlipOut

PopFlash

GlowPulse

RippleMask

SpiralIn

GlitchPop

BreathingIdle

Darker

RainbowFlash

WarmFlash

CoolFlash

Desaturate

NeonPulse

ColorCycle



Properties

All public properties and their descriptions



UiJuiceAnimator Properties

bool

autoPlayOnEnable

 If enabled, automatically plays Play IN when the object becomes enabled (Play Mode by default; optionally Edit Mode too).

Default: true

float

stagger

 Additional per-element delay (seconds) applied based on list order. Creates a cascading/staggered effect.

Default: 0.01f

bool

includeInactiveChildren

 When scanning children, include inactive UI objects too (useful for menus that start disabled).

Default: false

bool

autoRandomizeOnPlay

 If enabled, re-applies the current preset (with variation) right before playing (Play In). Adds randomness to keep animations fresh.

Default: true

float **strength** [Range (0.1f, 2.5f)]

👊 Intensity multiplier applied to the chosen feeling/preset. Higher = bigger offsets, punchier motion, shorter durations.

Default: 1.0f

float **durationMin** [Range (0.1f, 5f)]

🕒 Minimum duration (seconds) used by presets/variation. Element durations will be clamped to this range.

Default: 0.35f

float **durationMax** [Range (0.1f, 5f)]

🕒 Maximum duration (seconds) used by presets/variation. Element durations will be clamped to this range.

Default: 0.8f

FeelingPreset **preset**

🐱 The style/feeling to apply across elements (offsets, curves, timing, etc.). Choose from 60+ presets.

Default: FeelingPreset.Dramatic

bool **deactivateGameObjectAfterPlayOut**

If enabled, the GameObject will be deactivated when a Play OUT finishes (after the last element completes). Runtime only.

Default: false

bool

destroyGameObjectAfterPlayOut

If enabled, the GameObject will be destroyed when a Play OUT finishes (after the last element completes). Runtime only. Takes precedence over Deactivate.

Default: false

bool

IsAnimating { get; }

✓ Returns true if any animations are currently running. Useful for preventing overlapping animations.

UnityEvent

onAnimationStart

🎬 Invoked when animation starts (Play In or Play Out). Connect to other systems like sound effects or particle systems.

UnityEvent

onAnimationFinish

🎬 Invoked when all animations finish. Useful for triggering follow-up actions after animations complete.

List<JuiceElement>

elements

List of UI elements that will be animated. Use 'Scan Children' in the inspector to populate automatically, or add manually.

Default: new List<JuiceElement>()

JuiceElement Properties

General

`bool`

`ignore`

If enabled, this element will be ignored and not included in animations.

Default: false

`RectTransform`

`target`

The UI element (RectTransform) that this animation entry will control.

Position Animation

`bool`

`animatePosition`

If enabled, animates anchoredPosition using Position Offset (and optional curve/shake).

Default: true

`Vector2`

`positionOffset`

Where the element starts (relative to its initial position) when playing IN. When playing OUT, this becomes the end offset.

Default: new Vector2(0f, -80f)

`bool`

`useCurvePath`

Adds a curved arc to the position tween.

Default: true

float

curveAmplitude

Arc height for the curve path.

Default: 20f

bool

useShakeJitter

Enables high-frequency shaking jitter.

Default: false

float

shakeMagnitude

Magnitude of shake jitter in UI units.

Default: 6f

float

shakeFrequency

Shake oscillations per second.

Default: 20f



Rotation Animation

bool

animateRotation

If enabled, animates localRotation using Rotation Offset.

Default: false

`Vector3`

`rotationOffset`

Rotation applied at the start of Play IN (and at the end of Play OUT). Z is most commonly used for UI.

Default: new Vector3(0f, 0f, 12f)



Scale Animation

`bool`

`animateScale`

If enabled, animates localScale using Scale From.

Default: true

`Vector3`

`scaleFrom`

Scale at the start of Play IN (and at the end of Play OUT).

Default: new Vector3(0.85f, 0.85f, 1f)



Fade Animation

`bool`

`animateFade`

If enabled, animates a CanvasGroup alpha (adds one if missing).

Default: false

`float`

`fadeFrom`

Alpha at the start of Play IN (and at the end of Play OUT).

Default: 0f



Color Animation

`bool`

`animateColor`

If enabled, animates the Graphic color (Image/Text/TMP via Graphic) from Color From.

Default: false

`Graphic`

`graphicOverride`

Optional graphic to animate color on; defaults to target's Graphic.

`Color`

`colorFrom`

Color at the start of Play IN (and at the end of Play OUT).

Default: Color.white



Timing

`float`

`duration`

How long this element's tween takes (seconds).

Default: 0.45f

`float`

`delay`

Extra delay before this element starts (seconds). Stagger is added on top.

Default: 0f

AnimationCurve

ease

Easing curve for this element. X=time (0..1), Y=progress (0..1).

Default: JuiceMath.DefaultEase

© 2024 UI Juicer - One-Click UI Animator

Complete Documentation v1.0